Modelling Deadlock Resolution in Open Restricted Queueing Networks

Geraint I. Palmer-Liyu & Rhyd Lewis

October 28, 2026

1 Introduction

The simulation software used is the Ciw library in Python [3], although the methodology is appropriate for general DES.

2 Preliminaries and System Model

This work builds directly on the work of [2], which considered deadlock detection in discrete event simulations (DES) of open restricted queueing networks, and analysed Markovian models of deadlocking queueing system. This section summarises the findings of that work, defining concepts and notation and results as needed.

A service centre comprises a finite number of servers and a queueing buffer. Arriving customers enter service immediately if a server is free, otherwise they wait in the buffer subject to a given service discipline, or are turned away if the buffer is full. When there are multiple service centres, service centres are called nodes of a queueing network: when a customer completes service at a service centre, that is spends a random amount of time there, there is a probability that they will then arrive at another node. The network is open if customers can enter and leave from the exterior, and restricted if at least one node has finite capacity, causing externally arriving customers to be turned away and internally routed customers to be blocked when full.

There are three types of blocking, and this works considers Type I blocking only [1]: once a customer finishes their service and wishes to join another node's queue, they are blocked from joining that queue if it is full; while blocked that customer remains with its server until capacity becomes available at its destination; during this time that server is unavailable to begin another customer's service. This gives rise to the idea that one customer can be blocked by another customer: that is if customer A at node 1 is blocked from entering node 2 because it is full, then customer A is waiting on the customers who are with the servers at node 2 to exit, hence customer A is blocked by the customers who are with the servers of node 2. *Deadlock occurs when a subset of blocked customers are blocked by customers in that subset only.*

In [2] a graph theoretic approach to detecting deadlock during a simulation run is given:

Definition 1. *Let $\tilde{D}(t) = (\tilde{V}, \tilde{E}(t))$ be the state digraph of a queueing system, with set of vertices $\tilde{V}$ corresponding to the servers of all service centres, and a set of directed edges $\tilde{E}(t)$ representing blockage relationships such that if a customer at the kth server of node i is blocked from entering node j, then there are directed edges from the vertex corresponding to node i's kth server to every vertex corresponding to the servers of node j.*

Note that the vertex set $\tilde{V}$ can be partitioned into sets of servers all sharing the same node, such that $\tilde{V} = \tilde{V}_1 \cup \tilde{V}_2 \cup \cdots \cup \tilde{V}_m$, where $\tilde{V}_i$ contains all servers at node i. Therefore $\tilde{V} = \{\tilde{V}_1, \tilde{V}_2, \ldots \tilde{V}_m\}$.

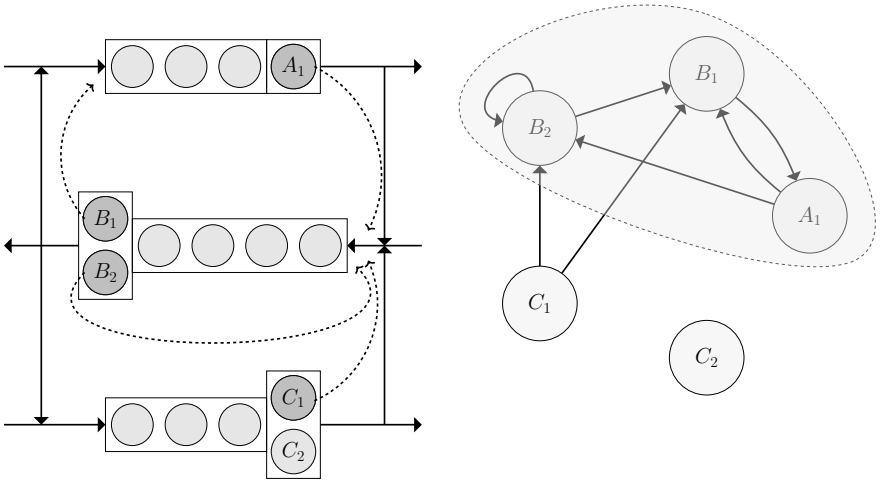


Figure 1: Example of a three node network in deadlock, and the state digraph $\tilde{D}(t)$ that describes the blocking relationships. Customers with servers A_1, B_2 and C_1 are blocked to node B, while the customer with server B_1 is blocked to node A. The knot $\tilde{K}(t)$ within $\tilde{D}(t)$ is highlighted - note that server C_1 is not part of the knot, as no deadlocked customer is waiting for this server to be free.

Then the following theorem is proved:

Theorem 1 (Palmer et al., 2018 [2])**.** *A deadlocked state arises at time t if and only if $\tilde{D}(t)$ contains a knot.*

A knot here is a strongly connected component with no sink or out-edges. Denote $\tilde{K}(t) \subseteq \tilde{D}(t)$ the subgraph that represents this knot, that is the subgraph induced by the set of vertices involved in the knot. Figure 1 shows an example of a three node queueing network in deadlock with the corresponding state digraph.

Note a difference in notation in this paper compared to [2], where we use $\tilde{D}$, $\tilde{V}$, $\tilde{E}$ and $\tilde{K}$ to denoted the graphs and sets described in that paper. Corresponding variables without tildes will describe new yet related constructions in this paper.

3 A New Construction

Here we define a new analogous construction to $\tilde{D}(t)$:

Definition 2. *Let $D(t) = (V, E(t))$ be an alternative state digraph, where V represents the set of nodes of the queueing network, and $E(t)$ represents blocked customers such that $(u_i, u_j) \in E(t)$ if a customer at node u_j is blocked to node u_i.*

Another way to achieve this construction is to apply a transformation Φ, defined by vertex contraction and edge reversal: vertices of $\tilde{V}$ in the same partition $\tilde{V}_i$ contract, and edges reverse and combine such that:

- if $(v_i, v_j), (v_i, v_k) \in \tilde{E}(t)$ and v_j and v_k contract to a new vertex v, then there exists the edge (v, v_i) in $E(t)$,
- if $(v_j, v_i), (v_k, v_i) \in \tilde{E}(t)$, and v_j and v_k contract to a new vertex v, then there exist *two* instances of the edge (v_i, v) in $E(t)$.

Lemma 1. *Let $\tilde{D}(t)$ be the state digraph of a queueing network as described in Definition 1, and $D(t)$ be the alternative state digraph of the same queueing network as described in Definition 2. Then $D(t) \equiv \Phi(\tilde{D})(t)$.*

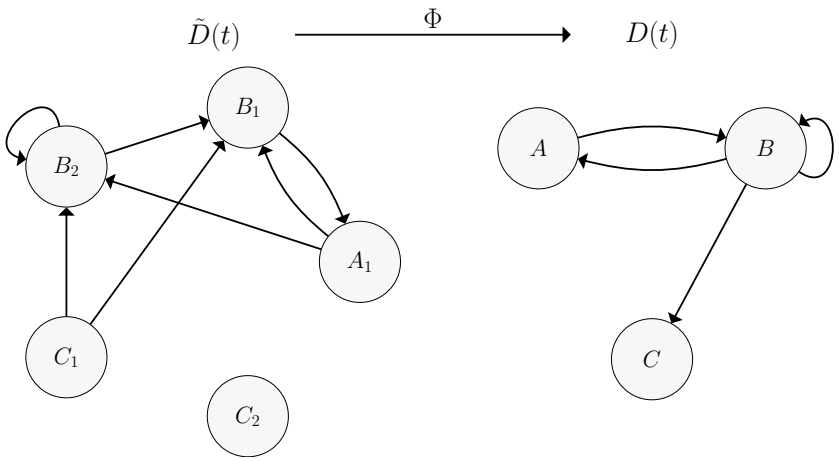


Figure 2: Example of the transformation Φ on the same state digraph as shown in Figure 1.

Proof. We demonstrate that the definition of $D(t)$ aligns precisely with the operations of Φ applied to $\tilde{D}(t)$ by showing that their vertex sets and edge sets correspond:

- *Vertices* ($V \equiv \Phi(\tilde{V})$): Elements of V corresponds to the nodes of the queueing network. Elements of $\tilde{V}$ correspond to servers of the network, and applying Φ contracts all servers at the same node, hence $\Phi(\tilde{V})$ also correspond to nodes of the queueing network.
- *Edges* ($E \equiv \Phi(\tilde{E})$): An edge $(v, u) \in E$ corresponds to a customer at node u being blocked to node v. An edge $(u, v) \in \tilde{E}$ corresponds to a customer at server u being blocked by server v, and applying Φ:
  - contracts edges originating from the same vertex and terminating at contracted vertices: these original edges corresponded to the same blocked customer being blocked by multiple servers at the same node, and so this contracted edge corresponds to a single unique blocked customer;
  - does not contract edges originating from contracted vertices and terminating at the same vertex: these original edges corresponded to two different customers being blocked by the same server;
  - reverses the direction of all edges after contraction, which does not alter edge counts or their interpretation;

  hence edges in $\Phi(\tilde{E})$ correspond to each blocked customer, originating from the vertex corresponding from the node to which they are blocked to, and terminating at the node they are currently at.

As $V \equiv \Phi(\tilde{V})$ and $E \equiv \Phi(\tilde{E})$, then $D(t) \equiv \Phi(\tilde{D})(t)$. □

For notational simplicity, we omit the time index (t) from now on. Figure 2 shows an example of applying Φ to an instance of $\tilde{D}$ to obtain the alternative state digraph D. Note in particular the reversal of the edges as they contract.

Theorem 2. *If $\tilde{K} \subseteq \tilde{D}$ is a subgraph induced by the knot in $\tilde{D}$, then $K = \Phi(\tilde{K})$ is also strongly connected.*

Proof. The mapping $\Phi(\tilde{K})$ is a transformation consisting of applying vertex contraction and edge reversal to the strongly connected graph $\tilde{K}$.

1. Vertex Contraction: Contracting the vertices of a strongly connected digraph preserves strong connectivity, as reachability between any pair of vertices is maintained.
2. Edge Reversal: Reversing all edges in a digraph preserves strong connectivity, as every directed path u to v in K becomes a valid path v to u in $\Phi(\tilde{K})$.

Therefore, $K = \Phi(\tilde{K})$ is strongly connected. □

It will also be useful to define one more subgraph:

Definition 3. *At deadlock, let $H_u = (V_{H_u}, E_{H_u})$, where $H_u \subseteq K = \Phi(K)$, be some Eulerian subgraph of the transformed knot $\tilde{K} \subseteq \tilde{D}$ that contains the vertex $u \in V_{H_u}$.*

As H_u is Eulerian, then:

- $\deg_{\text{in}}(v) = \deg_{\text{out}}(v)$ for all $v \in V_{H_u}$, and
- there exists an Eulerian circuit in H_u, that is a cycle that traverses each edge once and only once.

4 Sequences of Unblockages

At any time t during the simulation, some subset of the customers may be blocked. Define a blockage event as one specific instance of a specific customer being blocked, and write B_k as the kth blockage to occur during the simulation run. Now:

Definition 4. *Let $(B_n)_{n\in\mathcal{N}(t)}$ be the sequence of active blockage events at time t, where $\mathcal{N}(t) \subset \mathbb{N}$, and $|\mathcal{N}|$ is bounded above by the number of servers in the queueing network.*

Blockage events are associated with both individual customers and the servers who are occupied with the blocked customers. Recall that V, the set of vertices of the state digraph $D(t)$, represents the set of all servers, therefore there is a mapping between blockage events and servers:

Definition 5. *Let:*

- *$\nu_{at} : (B_n)_{n\in\mathcal{N}(t)} \to V$ be a mapping between blockage events and nodes, such that $\nu_{at}(B_i) \in V$ represents the node where the customer is currently blocked.*
- *$\nu_{to} : (B_n)_{n\in\mathcal{N}(t)} \to V$ be a mapping between blockage events and nodes, such that $\nu_{to}(B_i) \in V$ represents the node where the customer is blocked to.*

Let's consider how the system evolves over time t. Both the set of active blockages $\mathcal{N}(t)$ and the sequence $(B_n)_{n\in\mathcal{N}(t)}$ evolve with t. Let's also introduce $M(t)$, the total number of blockages that have occurred so far in the simulation run. Then with each blockage and each unblockage the system state is updated simultaneously by:

- When a customer at node u is blocked to node v:
  - The total blockage counter $M(t)$ increments by 1,
  - The active blockage set $\mathcal{N}(t)$ expands to $\mathcal{N}(t) \cup \{M(t) + 1\}$,
  - Create $B_{M(t)+1}$ such that $\nu_{\text{at}}(B_{M(t)+1}) = u$ and $\nu_{\text{to}}(B_{M(t)+1}) = v$,
  - The set of edges of the alternative state digraph $D(t)$ includes the new blockage relationship, so $E(t)$ becomes $E(t) \cup \{(v, u)\}$.
- When the blockage event B_k becomes unblocked:
  - the index leaves the active blockage set $\mathcal{N}(t)$ to become $\mathcal{N}(t) \setminus \{k\}$,
  - The associated edge is removed from the $D(t)$, so $E(t)$ becomes $E(t) \setminus \{(v, u)\}$.

Table 1 shows how a sequence of blockages and unblockages can lead to the deadlock seen in Figure 1, and how $(B_n)_{n\in\mathcal{N}(t)}$, $\mathcal{N}(t)$, $M(t)$, $\tilde{D}(t)$ and $D(t)$ evolve at each step. In this example there are three nodes: Node A with one server A_1, Node B with two servers B_1 and B_2, and Node C with two servers C_1 and C_2.

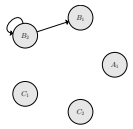
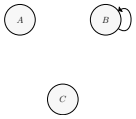
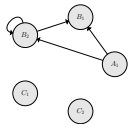
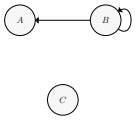
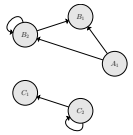
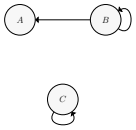
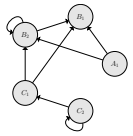
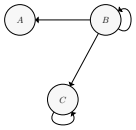
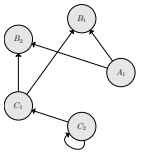
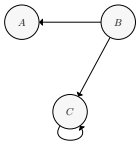
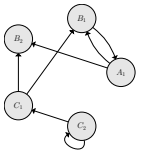
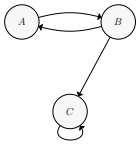
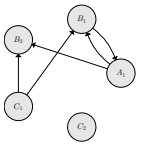
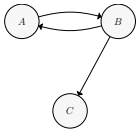
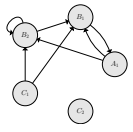
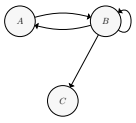
Event	$M(t)$	$\mathcal{N}(t)$	$(B_n)_{n\in\mathcal{N}(t)}$	$\tilde{D}(t)$	$D(t)$
Customer at B_2 is blocked to Node B	1	$\{1\}$	B_1		
Customer at A_1 is blocked to Node B	2	$\{1,2\}$	B_1, B_2		
Customer at C_2 is blocked to Node C	3	$\{1,2,3\}$	B_1, B_2, B_3		
Customer at C_1 is blocked to Node B	4	$\{1,2,3,4\}$	B_1, B_2, B_3, B_4		
Customer at B_2 is unblocked	4	$\{2,3,4\}$	B_2, B_3, B_4		
Customer at B_1 is blocked to Node A	5	$\{2,3,4,5\}$	B_2, B_3, B_4, B_5		
Customer at C_2 is unblocked	5	$\{2,4,5\}$	B_2, B_4, B_5		
Customer at B_2 is blocked to Node B	6	$\{2,4,5,6\}$	B_2, B_4, B_5, B_6		

Table 1: An example sequence of blockages with unblockages.

An unblockage can itself trigger further unblockages, for example if there exists a set of blockages $\{B_i, B_j, ...\}$ are with $\nu_{\text{to}}(B_i) = \nu_{\text{to}}(B_j) = \cdots = \nu_{\text{at}}(B_k)$, then unblocking B_k will allow one of $\{B_i, B_j, ...\}$ to become unblocked. This choice is governed by two conditions given in Remark 1.

Remark 1. *Unblocking B_k will also trigger the unblocking of B_h if:*

1. $(\nu_{to}(B_k), \nu_{at}(B_h)) \in E(t)$*, that is that B_h is blocked by B_k; and*
2. $h = \min(i \in \mathcal{N}(t) \mid (\nu_{to}(B_k), \nu_{at}(B_h)) \in E(t))$*, that is out of all the active blockages caused by B_k, B_h was the first to become blocked.*

Note that condition 1 is a necessity, an unblockage can only occur when there is spare capacity at the destination node, which is triggered by the original unblockage. Condition 2 meanwhile is a choice: it would be natural in a FIFO service discipline that those customers who have waited longest should be unblocked first, retaining the eventual queueing order at the destination node; though in other service disciplines other selection criteria may be more appropriate. As an example, Figure 3 shows a system with a sequence of blockages, and Table 2 shows the sequence of unblockages that would occur *instantaneously* when server B_1 releases their customer and frees capacity at Node B. This sequence of cascading unblockages eventually unblocks servers C_1, B_2 and A_1, while server C_2 remains blocked.

Notice how $D(t)$ evolves as the sequence of unblockages unfolds: first (B, C) is removed, then (C, B), and finally (B, A). This forms a path in $D(t)$ beginning at the node B where capacity is first introduced by releasing a customer, and ending at an eventual sink. This traces how the capacity, introduced by releasing a customer from node B, is transferred from node to node via the sequence of unblockages. Lemma 2 formalises this.

Lemma 2. *A sequence of unblockages caused by capacity becoming available at a node $v \in V$ corresponds to a path in $D(t)$ starting at v and terminating at a vertex with no more unused out-edges.*

Proof. Let $\mathcal{P}$ be the path.

If capacity becomes available at Node $v \in V$, and there is at least one customer blocked to node v, then at least once customer blocked to node v will become unblocked. Without loss of generality, let the customer to be unblocked be B_n, and $\nu_{\text{at}}(B_n) = u$. Then $\nu_{\text{to}}(B_n) = v$, and Condition 1 the edge (v, u) exists in $E(t)$.

This unblockage moves a customer from u to v, and hence moves capacity from v to u, corresponding to adding edge (v, u) to $\mathcal{P}$.

If any $(u, w) \in E(t)$ exist, for any $w \in V$, then there is a customer at w blocked to u who can now be unblocked, and so this repeats, continuing the path.

The sequence of unblockages terminates if there are no customers blocked to u, and the path terminates at u. In this case there cannot exist any unused $(u, w) \in E(t)$ for any $w \in V$, and so u has no unused out-edges, and $\mathcal{P}$ terminates at a vertex with no more unused out-edges. □

To illustrate the arbitrariness of condition 2 above, consider the case where server A_1 was the first to get blocked. Once server B_1 releases their customer and creates capacity at Node B, A_1 is unblocked, but as there are no other blockages caused by A_1, condition 1 is not satisfied, and so there are no more unblockages. Hence servers B_2, C_1 and C_2 all remain blocked.

TODO

Show that this is the true protocol for Ciw and SIMUL8. Do this (in an Appendix) by designing a deterministic system where the sequence of unblockages does indeed change macro behaviour: show we get the same results in Ciw as we do in SIMUL8.

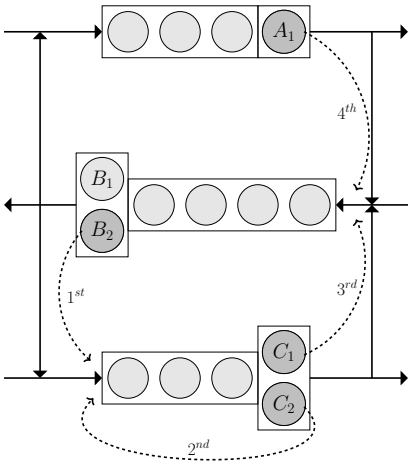


Figure 3: Example of a sequence of blockages and corresponding state digraph $D(t)$, for which Table 2 shows the sequence of unblockages when the customer at server B_2 exits the system.

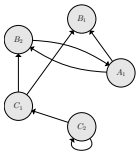
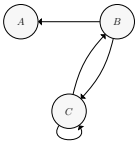
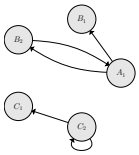
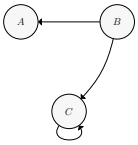
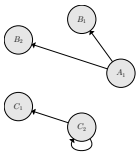
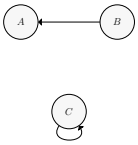
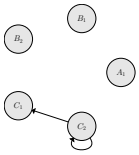
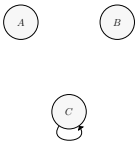
Event	$M(t)$	$\mathcal{N}(t)$	$(B_n)_{n\in\mathcal{N}(t)}$	$\tilde{D}(t)$	$D(t)$
Customer at B_1 is released	4	$\{1,2,3,4\}$	B_1, B_2, B_3, B_4		
Customer at C_1 is unblocked to Node B	4	$\{1,2,4\}$	B_1, B_2, B_4		
Customer at B_2 is unblocked to Node C	4	$\{2,4\}$	B_2, B_4		
Customer at A_1 is unblocked to Node B	4	$\{2\}$	B_2		

Table 2: An example sequence of cascading unblockages.

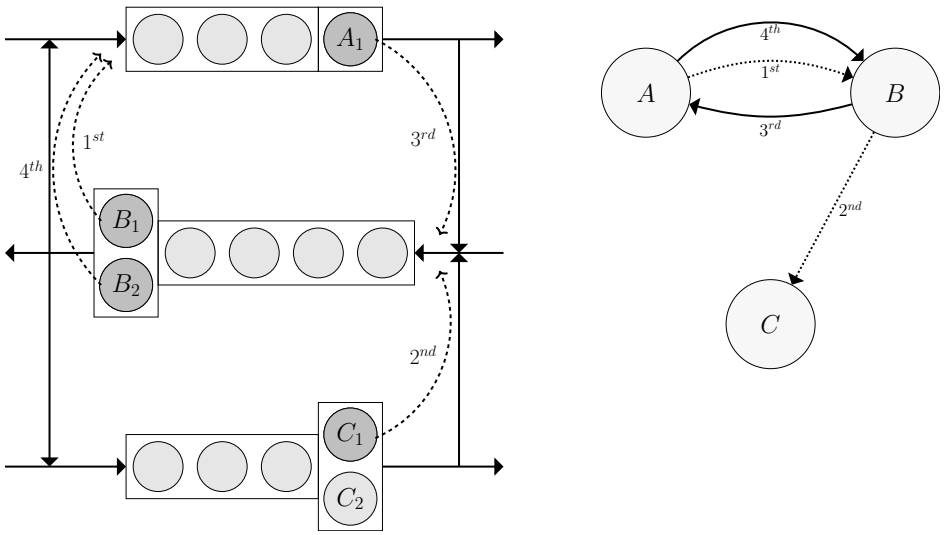


Figure 4: Example of a deadlock where the induced sequence of unblockages by adding dummy capacity at Node A would not be feasible under the conditions set out in Remark 1. On the right hand side is a depiction of $D(t)$, with the corresponding unblockage path given with dotted edges.

5 Deadlock Resolution

5.1 Induced Sequence of Unblockages

For a cascading sequence of unblockages, which 'untangles' the blockages built up over time, to occur, it is necessary for the customer at the head of the sequence to be blocked by servers who have the potential to create capacity at their node. That is, blocked by servers that are themselves are not blocked, or that the system is not deadlocked. We propose introducing some artificial or dummy capacity as a node in order to *induce* a sequence of unblockages that will untangle the deadlock.

Definition 6. *At deadlock, a sequence of unblockages is* INDUCED *by temporarily creating extra capacity for one customer at one of the nodes, u.*

The induced sequence of unblockages is FEASIBLE *if by the end of the sequence that extra capacity is not in use, and can therefore be removed.*

As sequences of unblockages happen instantaneously with respect to simulation time, a feasible induced sequence of unblockages will not cause any customer to experience any non-zero duration of time at the dummy capacity, and the nodes themselves will not experience any non-zero duration of time with extra capacity. Therefore this feasible induced sequence of unblockages plays no role in the modelled system, and is purely a simulation mechanic that stands as a proxy for mutual communication between deadlocked service centres.

Remark 2. *Not all induced subsequences are feasible under the conditions set out in Remark 1.*

The claim in Remark 2 can be shown with a simple counter-example: consider the three node queueing network in deadlock given in Figure 4, and consider inducing a sequence of unblockages by adding dummy capacity to Node A. Then customer at B_1 will become unblocked, filling the dummy capacity and leaving capacity at Node B; which in turn unblocks the customer at C_1, filling the capacity at Node B and leaving capacity at Node C. However no customer was originally blocked by C_1, and so no more unblockages can occur, and the dummy capacity introduced at Node A is still used. Hence this is not a feasible induced sequence of unblockages.

Lemma 3. *A sequence of unblockages induced by introducing artificial capacity at node u is feasible if and only if it corresponds to a path in $D(t)$ that is a cycle that originates and terminates at u.*

Proof. By Lemma 2 a sequence of unblockages which begins by capacity becoming available at node u traces a path $\mathcal{P}$ in $D(t)$; which is interpreted as the free capacity itself travelling along the path from node to node.

Introducing artificial capacity at u temporarily alters its queue capacity.

If the sequence is feasible then the system returns to a valid state that doesn't violate the system's original capacity constraints, and so the artificial capacity eventually propagates back to u. Therefore u features in $\mathcal{P}$ once again. Now as this artificial capacity is filled no more further unblockages will be triggered, therefore $\mathcal{P}$ terminates, and so it is directed cycle originating and terminating at u.

Conversely, if $\mathcal{P}$ is a cycle that originates and terminates at u, then the artificial capacity has propagated back to u, and so the induced sequence of unblockages is feasible. □

To avoid a situation such as this, it will be required to constrain the sequence of unblockages to ensure that $\mathcal{P}$ is a cycle.

5.2 Constrained Induced Sequence of Unblockages

Now we define a constrained induced sequence of unblockages. We have already set out in Lemma 2 that a sequence of unblockages corresponds to a path in $D(t)$, and that a sequence of unblockages induced by introducing artificial capacity at u corresponds to a path originating at u. Therefore we can define a constrained sequence of unblockages by constraining its corresponding path:

Definition 7. *Define a constrained induced sequence of unblockages caused by introducing artificial capacity at node u by the rules that unblocking B_k will also trigger the unblocking of B_h if:*

1. $(\nu_{to}(B_k), \nu_{at}(B_h)) \in E_{H_u}$*, that is that B_h is blocked by B_k; and*
2. $h = \min(i \in \mathcal{N}(t) \mid (\nu_{to}(B_k), \nu_{at}(B_h)) \in E_{H_u})$*, that is out of all the active blockages caused by B_k, B_h was the first to become blocked.*

Note that these rules are the same as those introduced in Remark 1, however we constrain the edges traversed to those in H_u, defined in Definition 3, rather than the full edge set of $D(t)$.

Theorem 3. *A constrained sequence of unblockages induced by introducing artificial capacity at node u is feasible.*

Proof. From Lemma 3, the path $\mathcal{P}$ corresponding to a feasible sequence of unblockages induced by introducing artificial capacity at u is a cycle originating and terminating at u. Definition 7 constrains unblocking customers to those corresponding to edges in H_u, while Condition 2 of that definition determines the choice of edge to traverse whenever there is a choice: that is choosing the customer who has been blocked longest. Hence the result should hold for any arbitrary ordering.

As H_u is Eulerian, then $\deg_{\text{in}}(v) = \deg_{\text{out}}(v)$ for every $v \in V_{H_u}$. If the path $\mathcal{P}$ enters node v then it 'consumes' an in-degree, and if it exits node v then it 'consumes' an out-degree, and the degree parity is maintained. $\mathcal{P}$ originates at u, then $\deg_{\text{out}}(u) > 0$, and therefore $\deg_{\text{in}}(u) > 0$ also. Each time it visits a vertex $v \neq u$ it is always able to escape that vertex, due to degree parity, hence it cannot ever get stuck at some $v \neq u$. Therefore the path will always find its way back to u, where it can terminate at u as a cycle. Hence its corresponding sequence of unblockages is feasible. □

5.3 Choosing an Eulerian Subgraph

TODO

Note that what we ideally want is a maximum cardinality minimum weight Eulerian subgraph. However - this is NP-Hard (cite this).

- Framework A: First we will use a greedy algorithm to find a maximal cardinality circuit beginning and ending in u - which is Eulerian by definition.
- Then we could possibly add the minimum weight objective?
- Framework B: Finally, we can consider changing the *ordering* of the edges so that the constrained unblockaged does indeed follow the circuit - ensuring all edges are traversed, hence all possible unblocked customers to become unblocked.

6 Inter-Deadlock Times

TODO

Once it's all up and running, we can run a simulation with automated deadlock detection and resolution. We are interested in T, the inter-deadlock time distribution, the distribution of durations between two consecutive deadlocks. Experiments to run:

- Relationship between T and queueing network parameters (in particular recreate the experiments from first paper).
- Relationship between T and time-to-first-deadlock (considered in first paper).
- Look at the 'gap' between T under Frameworks A and B - does queueing network parameters affect the gap?
- Consider large networks: N nodes each with c servers, equal transition probability everywhere. How does T (and gap) change as N or c increases? What if not all nodes are connected? Randomly generate a strongly connected network using some connectivity parameter (low = simple cycle; high = complete graph), how does T change with network connectivity?

(Possibly rather than look at inter-deadlock times, we can make it scale invariant, and look at number of simulation events between deadlocks...?)

7 Discussion

7.1 Other Blocking Mechanisms

This paper is concerned with Type I blocking only. Here we discuss why this framework would not work for blocking of Types II and III. First, recall the differences in the three types of blocking [1]:

- Type I blocking: here a customer is blocked after completing service.
- Type II blocking: here a customer declares their destination before beginning service, and is only granted service if there is available capacity at their destination node, that is they are blocked before service.
- Type III blocking: here a customer is required to repeat their service if there is no capacity at the destination node, and their destination is re-sampled.

At first glance, Type II and Type I blocking are similar, in that destinations are fixed. The key difference is that at an unblockage, for Type I blocking the movement is immediate, whereas for Type II blocking the unblocked customer now has to experience their service time before moving to the destination node. This means that the next downstream blockage is delayed; hence sequences of unblockages do not happen

immediately, as the service time delay propagates through the sequence. As such, a feasible induced sequence of unblockages under Type II blocking will not be immediate, and so the artificial capacity introduced to induce the sequence will not immediately disappear, violating the rules of the queueing network.

A similar situation might occur under Type III blocking, as the service is repeated at blockage, meaning those in deadlock will be perpetually repeating their service until some intervention. However a further complication arises here too: as the blockage destinations are constantly re-sampled, then $D(t)$ is probabilistic, where edges have particular probabilities of existing at any point. Therefore it cannot be guaranteed that the path $\mathcal{P}$ associated with the induced sequence of unblockages by adding artificial capacity at node u ever returns to u, as edges may cease to exist by the time the unblockages occur.

References

[1] Raif O Onvural and HG Perros. On equivalencies of blocking mechanisms in queueing networks with blocking. *Operations Research Letters*, 5(6):293–297, 1986.

[2] Geraint I Palmer, Paul R Harper, and Vincent A Knight. Modelling deadlock in open restricted queueing networks. *European Journal of Operational Research*, 266(2):609–621, 2018.

[3] Geraint I Palmer, Vincent A Knight, Paul R Harper, and Asyl L Hawa. Ciw: An open-source discrete event simulation library. *Journal of Simulation*, 13(1):68–82, 2019.